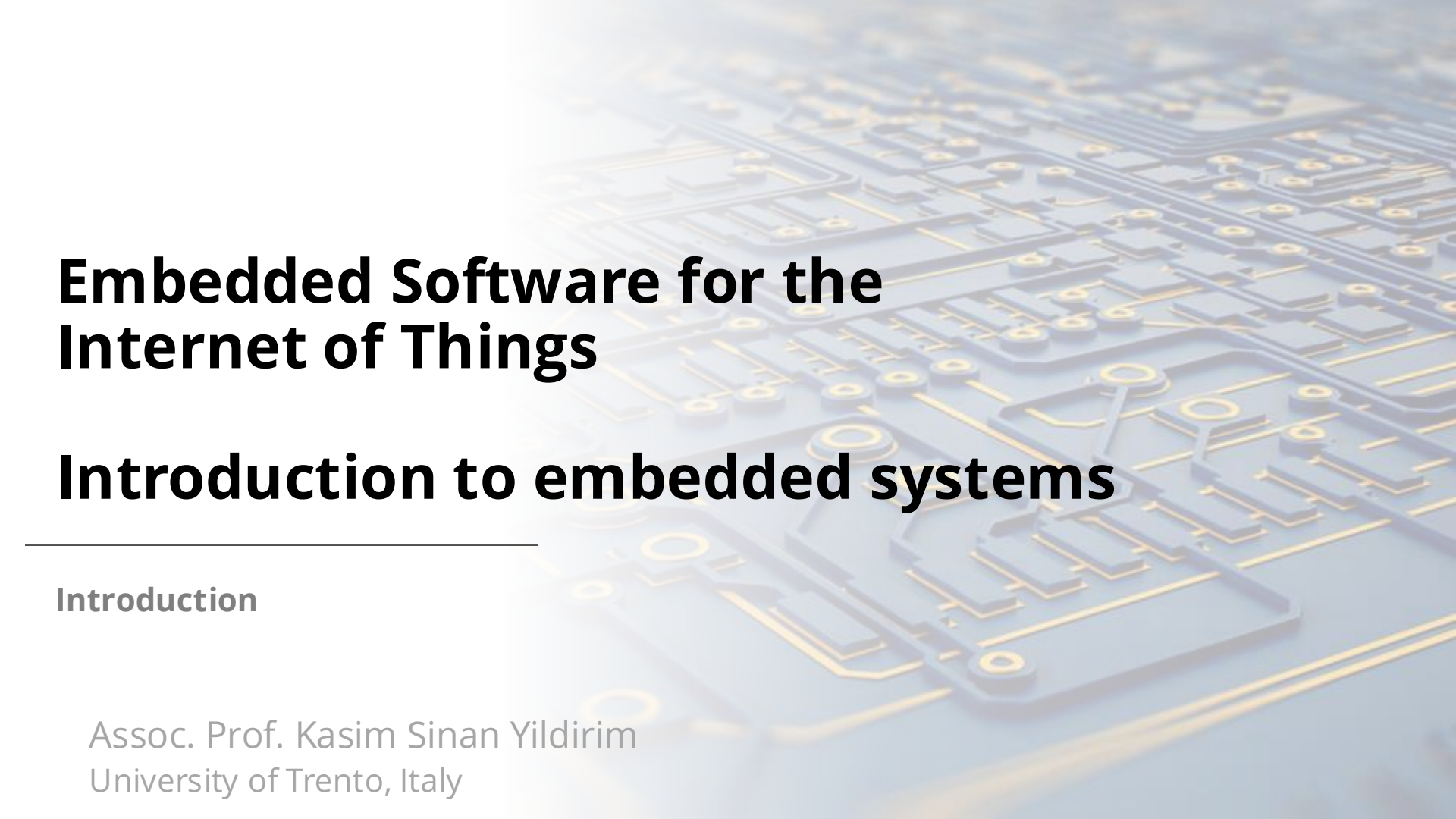




Embedded Software for the Internet of Things

Introduction to embedded systems

Introduction

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Our Course

- Embedded Software for the Internet of Things
 - Computer Science program
 - Computer, Communication and Electronic Engineering program
 - *Computer engineering area*
- Introduction to embedded systems
 - Computer, Communication and Electronic Engineering program
 - *Electronic engineering area*

Who am I?

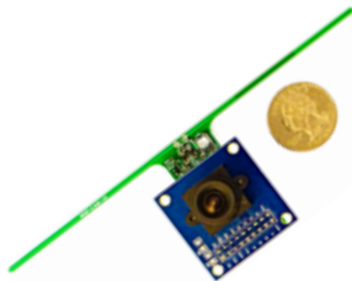
- Associate Professor@DISI, University of Trento
 - Call me "**Sinan**".
- Researcher on embedded systems
 - "Where the theory can meet the practice."

Who am I?

- Future sensing devices are tiny, sustainable, smart, and run forever!
 - Energy Harvesting
 - *Environment monitoring...*
 - Embedded AI Applications
 - *Smart wearables...*



Ambient Energy Sources



Batteryless Devices



Course Objectives

- Acquire the theoretical and practical skills to interface with state-of-the-art embedded computing platforms.
- Learn the fundamentals to design and develop IoT applications:
 - programming languages
 - hardware platforms
 - operating systems
 - software techniques, platforms and tools

Prerequisites

- We do not have any hard prerequisites.
- Background on the following topics is preferable:
 - Programming in C & Software Design
 - Assembly language (ARM)
 - Computer Architectures
 - Operating Systems
 - Basic Electronics

Course Contents

- Development Environments for Embedded Systems
- Embedded Software Design, Implementation and Testing
- Microcontroller-based IoT Platforms
 - Texas Instruments MPS432
- IoT Applications
- Basic Real-time Scheduling and Embedded RTOS

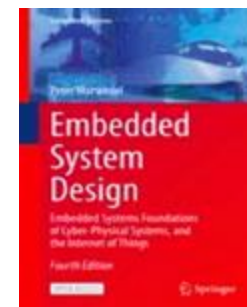
Course Contents (Practice)

- Bare Metal Programming
 - Microcontroller Interfacing, Ports and I/O
 - Polling/Interrupt Management
 - Timers and timer services
 - Analog to Digital Converters (ADC), Direct Memory Access (DMA)
 - Serial Communication (UART), SPI, I2C
- Sensor interfacing
 - Accelerometer, temperature, light, ...
- Application examples
- Programming Embedded RTOS



Textbook and Materials

- Course web site is sufficient:
 - <https://didatticaonline.unitn.it/dol/enrol/index.php?id=41973>
 - *Lecture Slides + Materials (Recorded Lectures)*
- Several books as supplemental text:
 - Computers as Components: Principles of Embedded Computing System Design, 1st Edition, Marilyn Wolf, Morgan Kaufmann
 - Real-Time Concepts for Embedded Systems, Qing Li, Caroline Yao, Taylor & Francis
 - Embedded System Design, Embedded Systems Foundations of Cyber-Physical and the Internet of Things, 4th Edition, Marwedel, P., Springer



Lectures Organization

- We will have mixed lectures:
 - There will be lectures covering **theory**
 - There will be lectures focused on **practice**
 - *Code and application examples...*
 - *You can come with your laptop.*
 - *You can form a pair of two students.*

Evaluation

- 35% Written Exam (Theory)
- 65% Project Evaluation
 - We will form **project groups** since we have a limited number of hardware kits:
 - *Groups of 4 students: Build a team for your project!*
 - You will design, implement, and demonstrate an **IoT application**.
 - *Presentation to the instructor (Source code, instructions, demo video, ...)*

Selected Projects

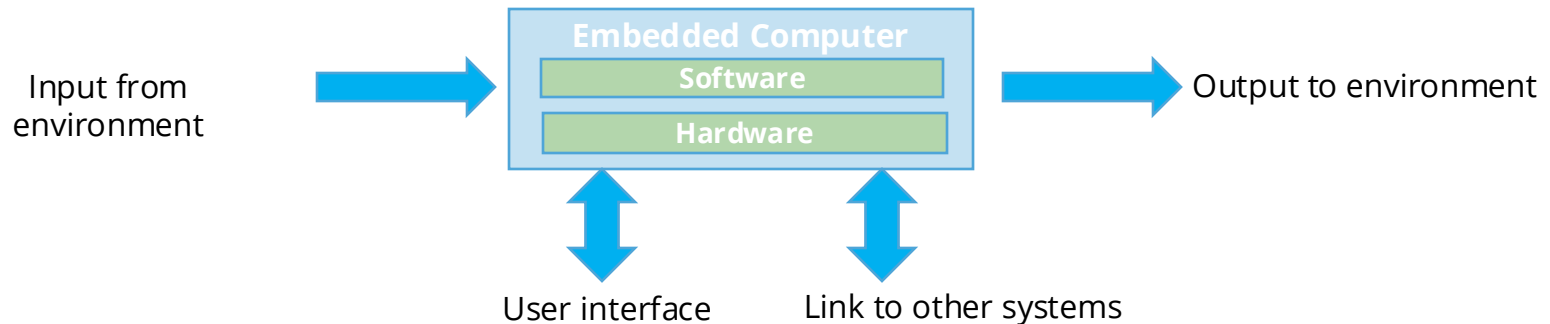
- [Konfort](#)
- [Smart MSP](#)
- [Claw Machine](#)
- [Sun Scanner](#)
- [Oscilloscope](#)
- [Receiver & Transmitter on RC-CAR](#)
- [Cube Solver](#)
- [Door Locking](#)
- [Tamagotchi](#)

Embedded Systems: Definition

- Computing systems with **tightly coupled** hardware and software integration:
 - Designed to perform a dedicated functionality
- The word “**embedded**” means:
 - Built into (to be an integral part of) a larger system
 - The larger system is called the embedding system.

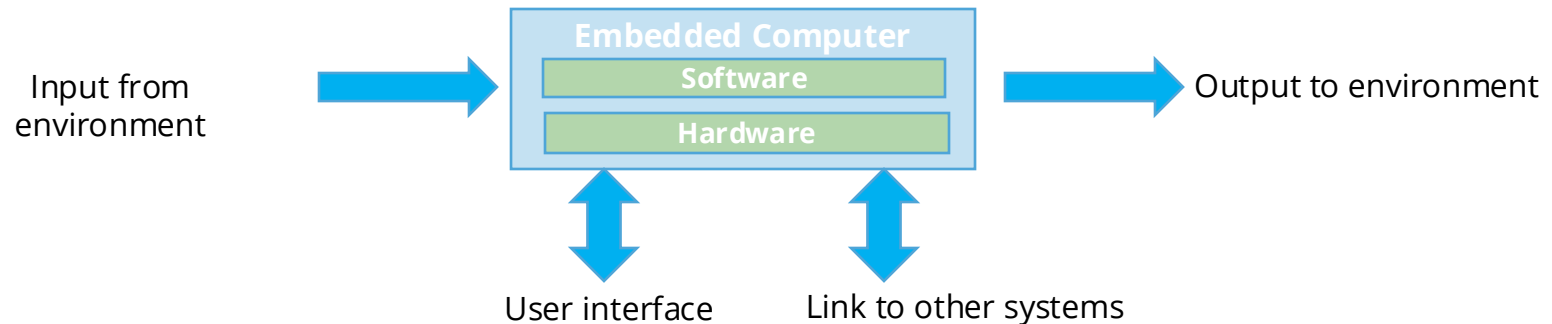
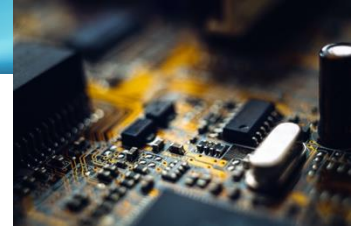
Embedded Systems: Definition

- **Application-specific** computer system
 - Often with real-time computing constraints
 - *How would you define **real-time**?*
- Typically implemented using microcontrollers (**MCUs**)
 - What is a microcontroller?



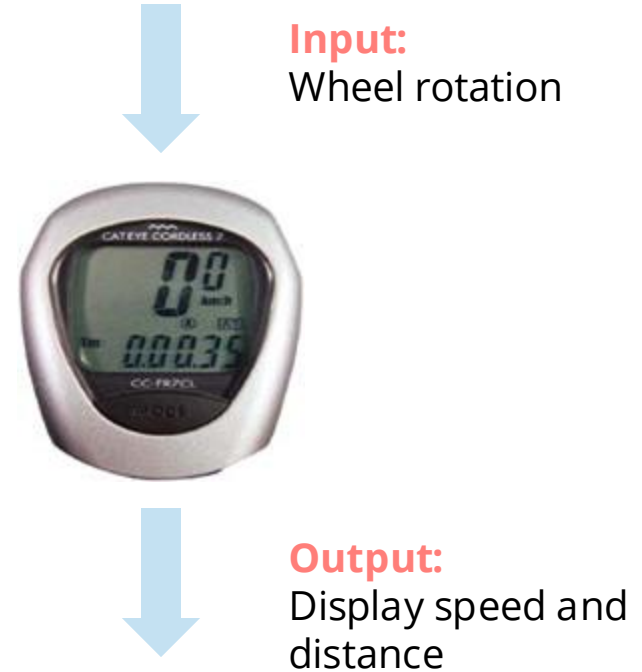
Embedded Systems: Definition

- Why add an embedded computer to a larger system?
 - Lower cost and performance, more functions and features, more dependability...



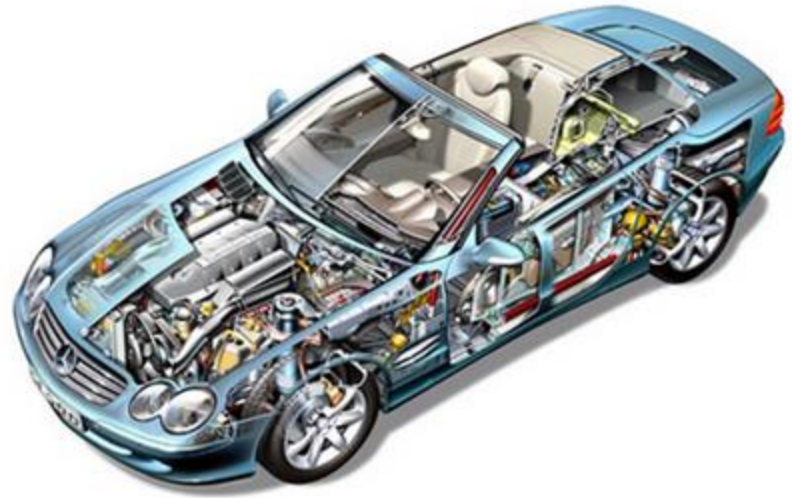
Example Embedded System: Bike Computer

- Functions
 - Speed and distance measurement
- Constraints
 - Size, Cost, Power and Energy, Weight
- Inputs
 - Wheel rotation indicator: how fast we are
- Output
 - Liquid Crystal Display
- Use Low-Performance Microcontroller
 - 8-bit, 10 MIPS



Example: Automotive Embedded systems

- Today's high-end automobile may have **100 microprocessors**:
 - 4-bit microcontroller checks seat belt;
 - Microcontrollers run dashboard devices;
 - 16/32-bit microprocessor controls the engine.
- Low-end cars use 20+ microprocessors.



Microcontroller vs. Microprocessor

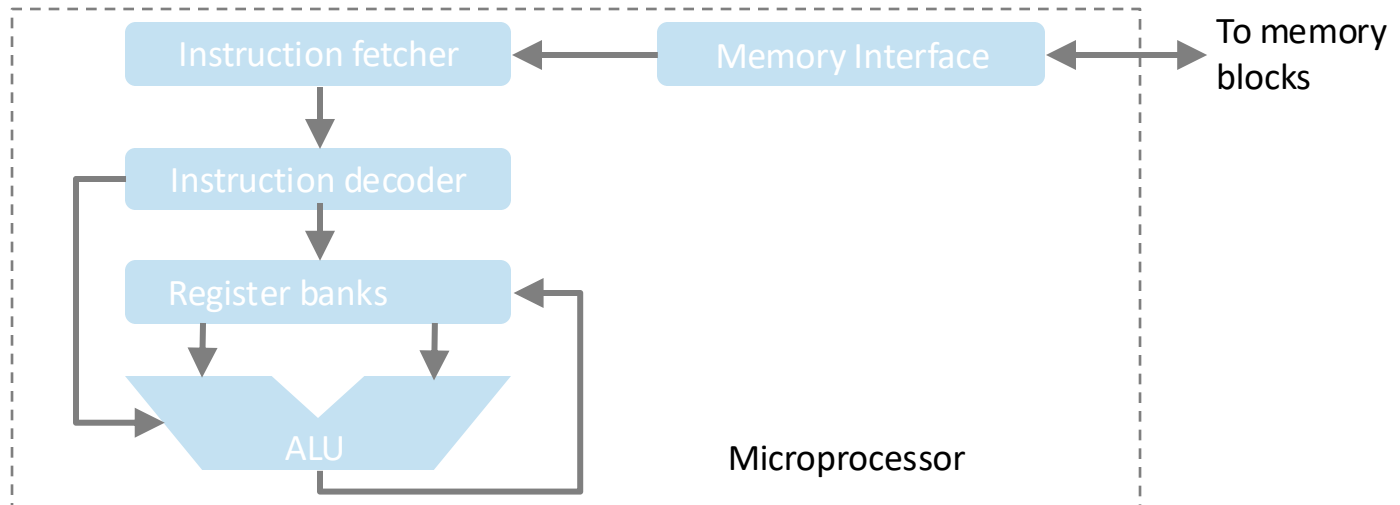
- What is the difference between an MCU and a CPU?

Microcontroller vs. Microprocessor

- What is the difference between an MCU and a CPU?
 - Both have a core to execute instructions

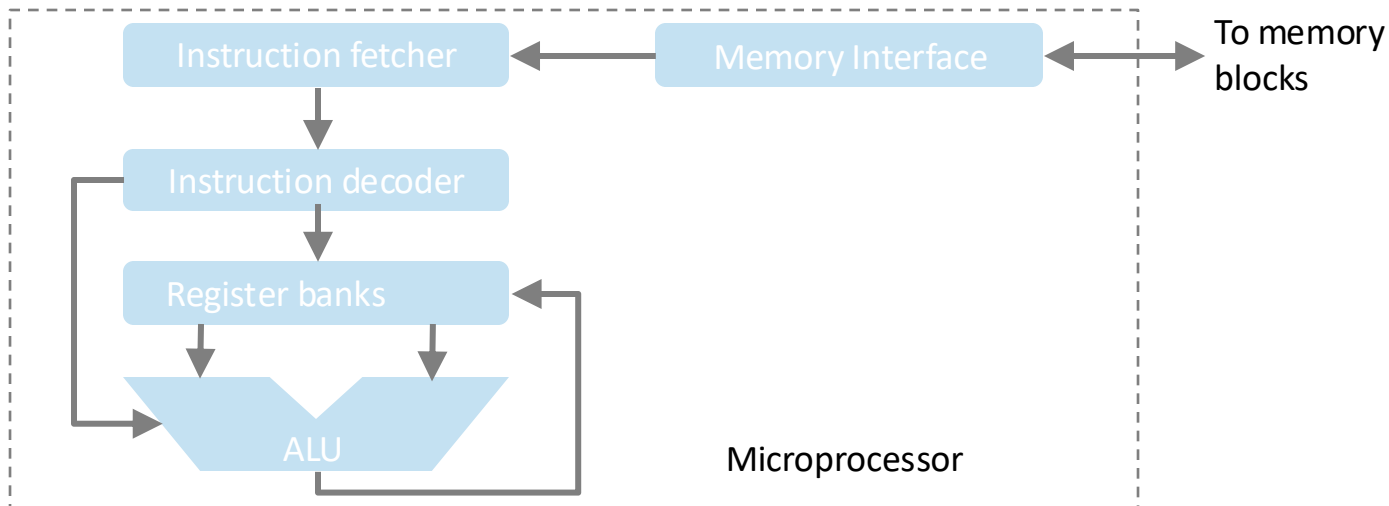
Microcontroller vs. Microprocessor

- CPU (Microprocessor)
 - A single processor core that supports at least instruction fetching, decoding, and executing
 - Used for general-purpose computing, but needs memories and Input/Outputs (IOs)



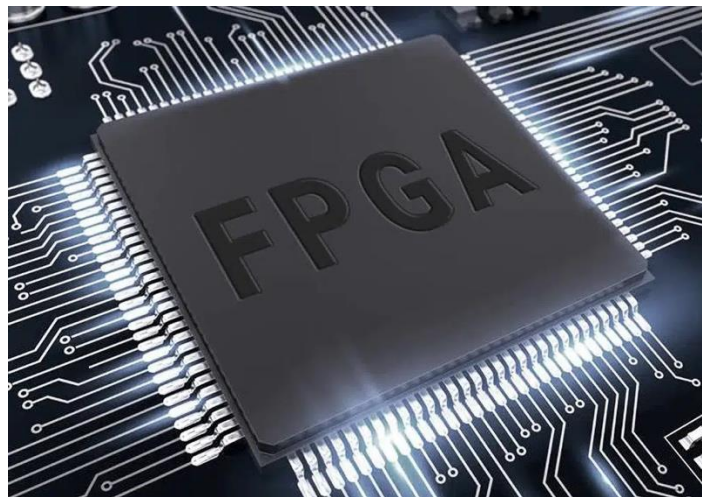
Why use Microprocessors?

- Microprocessors use the same hardware logic to perform many different functions:
 - Change the program it executes to run different algorithms
 - They simplify the design of families of products.



Why use Microprocessors?

- Alternatives: field-programmable gate arrays (FPGAs), custom logic, etc.
 - Dedicated hardware that has a special purpose:
 - *designed for a particular function*
 - It cannot be used for other functions.

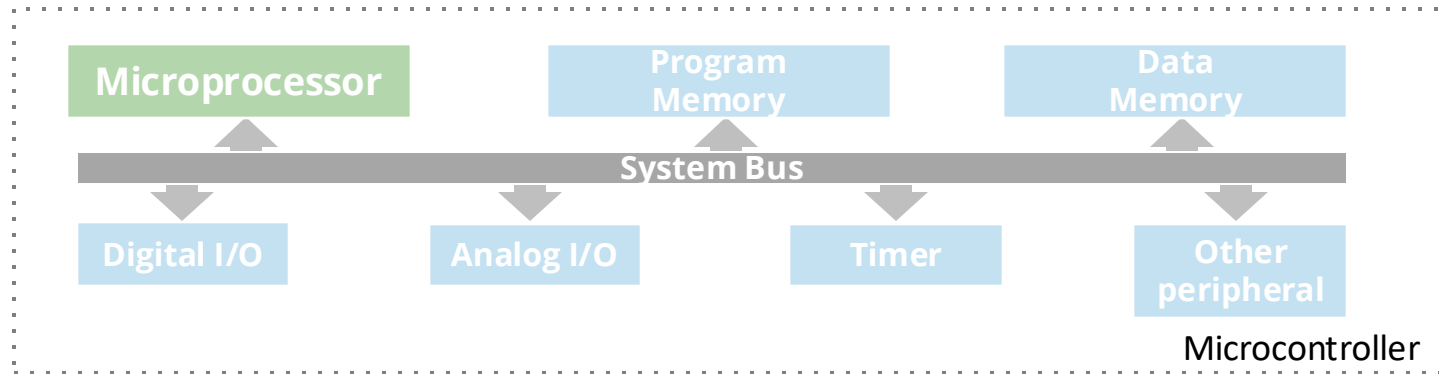


Why use Microprocessors?

- Modern microprocessors also offer features to control **power consumption**:
 - Software design techniques can help reduce power consumption.
- Heterogeneous systems
 - Some custom logic for well-defined functions
 - CPUs and software for everything else.

Microcontroller vs. Microprocessor

- MCU (Microcontroller)
 - Typically has a single processor core
 - Typically used for basic **control** purposes
 - Has memory blocks, Digital IOs, Analog IOs, and other basic peripherals



Microcontroller vs. Microprocessor

- In most embedded systems, MCUs are chosen to be the best solution, since they offer:
 - Low development and manufacturing cost
 - Easy porting and updating
 - Relatively low power consumption
 - Satisfactory performance for low-end products

Options for Building Embedded Systems

		Implementation	Design Cost	Unit Cost	Upgrades & Bug Fixes	Size	Weight	Power	System Speed
Dedicated Hardware		Discrete Logic	low	mid	hard	large	high	?	very fast
		ASIC	high (\$500K/ mask set)	very low	hard	tiny - 1 die	very low	low	extremely fast
		Programmable logic – FPGA, PLD	low	mid	easy	small	low	medium to high	very fast
Software Running on Generic Hardware		Microprocessor + memory + peripherals	low to mid	mid	easy	small to med.	low to moderate	medium	moderate
		Microcontroller (int. memory & peripherals)	low	mid to low	easy	small	low	medium	slow to moderate
		Embedded PC	low	high	easy	medium	moderate to high	medium to high	fast

Embedded System Functions

- Monitoring Applications (**Closed-loop** control system)
 - Monitor a process, and adjust an output to maintain desired set point (temperature, speed, direction, etc.)
- Sequencing
 - Step through different stages (**states**) based on environment and system
- Digital signal processing
 - Remove noise, select desired signal features
- IoT Applications (Communications and networking)
 - Exchange information reliably and quickly

Challenges in Embedded System Design

- How much hardware do we need?
 - How fast is the CPU? Memory size?
- How do we meet our deadlines?
 - Faster hardware or cleverer software?
- How do we minimize power?
 - Turn off unnecessary logic? Reduce memory accesses?

Challenges in Embedded System Design

- Does it really work?
 - Is the specification correct?
 - Does the implementation meet the specifications?
 - How do we test for real-time characteristics?
 - How do we test on real data?
- How do we design for upgradeability?
 - How to add features by changing software and hardware?
- Is it secure?
 - How are the dangerous acts prevented?

Challenges in Embedded System Design

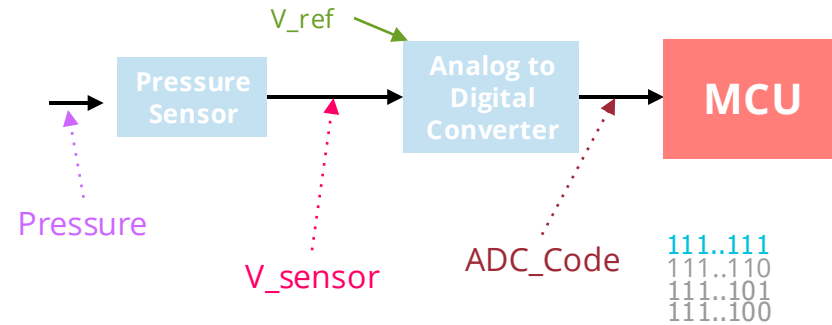
- Complex testing:
 - We cannot separate the testing of an embedded computer from the machine in which it is embedded.
 - *How to test a washing machine software?*
- Limited observability and controllability:
 - Difficult to see what is going on
 - *no keyboards and screens, need to watch the values of electrical signals*
- Restricted development environments:
 - Much more limited than those available for PCs

Properties of Embedded Systems

- Interfacing with the environment:
 - **Analog signals** from sensors: e.g., voltage value that represents a physical value
- **Concurrent** and **reactive** behaviours:
 - Respond to events **timely** (real-time constraints on responses)
 - Must perform multiple separate activities **concurrently** (e.g., several sensors...)
- Fault handling
 - Operate independently for long periods and handle likely faults without crashing
- Diagnostics
 - Help developers determine problems quickly

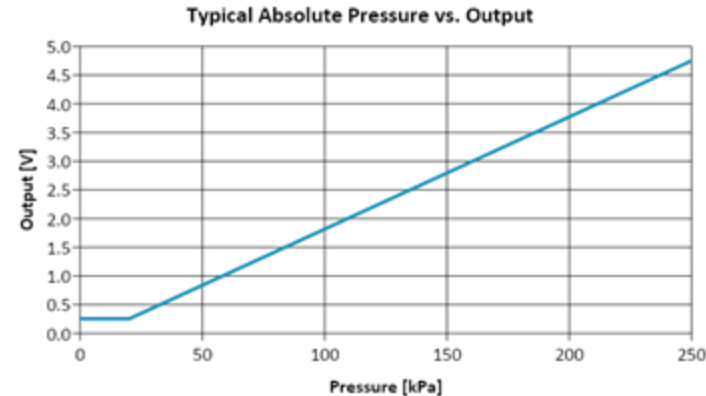
Example Analog Sensor

- We calculate how deep an underwater object is:
 - A sensor detects **pressure** and generates a proportional **output voltage** V_{sensor}
 - Analog to Digital Converter generates a proportional digital integer (code) based on V_{sensor} and V_{ref}



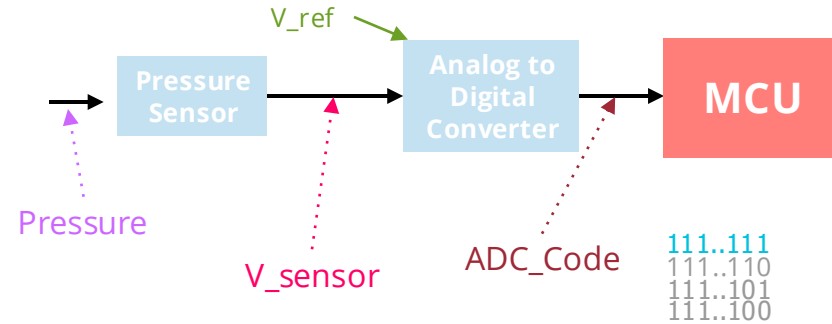
Software on MCU

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
```



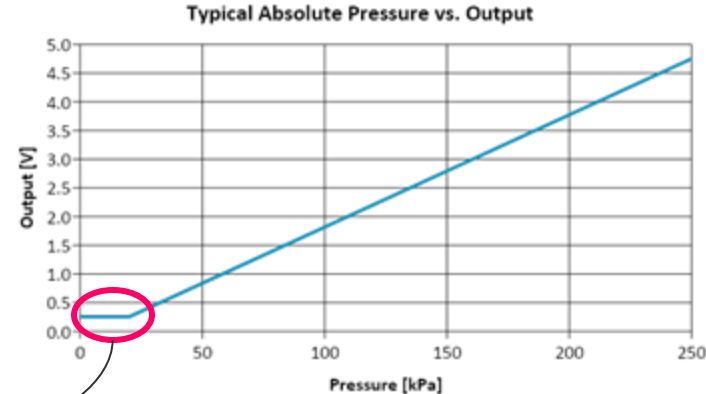
Example Analog Sensor

- We calculate how deep an underwater object is:
 - A sensor detects **pressure** and generates a proportional **output voltage** V_{sensor}
 - Analog to Digital Converter generates a proportional digital integer (code) based on V_{sensor} and V_{ref}



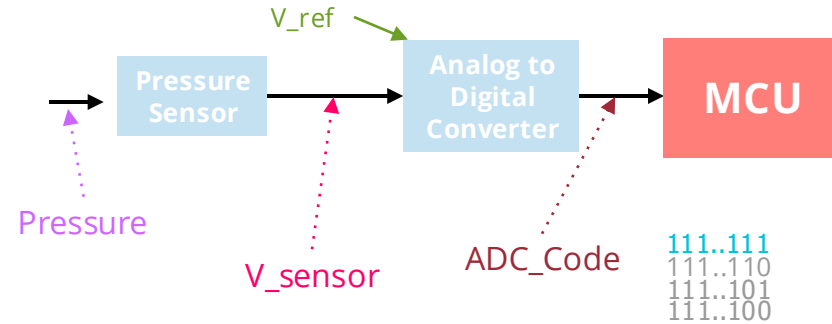
Software on MCU

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
```



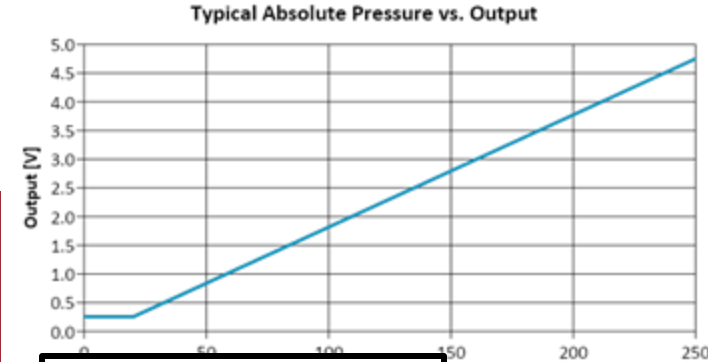
Example Analog Sensor

- We calculate how deep an underwater object is:
 - A sensor detects **pressure** and generates a proportional **output voltage** V_{sensor}
 - Analog to Digital Converter generates a proportional digital integer (code) based on V_{sensor} and V_{ref}



Software on MCU

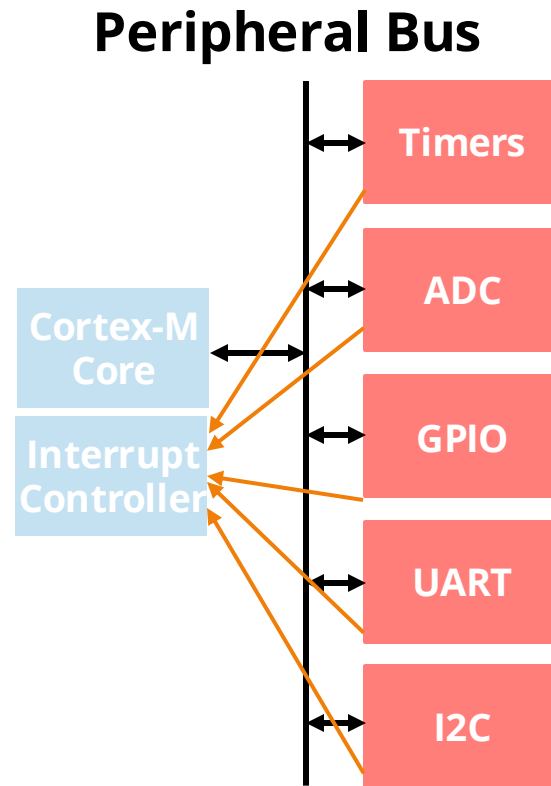
```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```



Actual relationship between depth and pressure

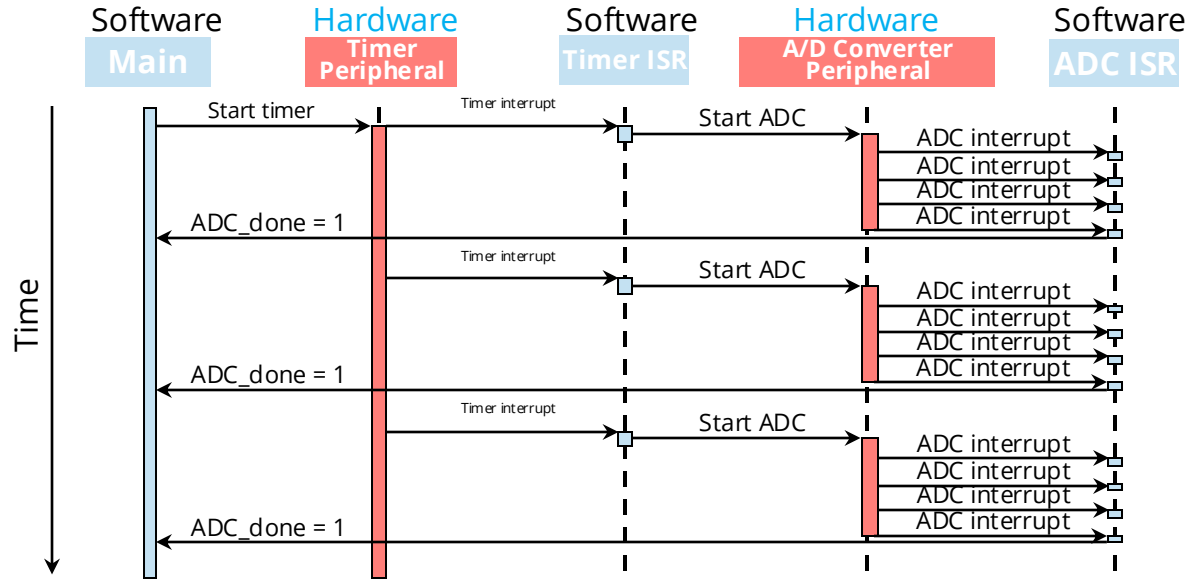
MCU Hardware & Software for Concurrency

- Things are happening **concurrently** while the CPU is executing instructions:
- Specialized hardware peripherals add dedicated concurrent processing
 - Analog interfacing, timers, detecting external signal events
- Peripherals use interrupts to notify the CPU about events



MCU Hardware & Software for Concurrency

- Embedded systems rely on both MCU hardware peripherals and software to get everything done concurrently on time.



Interrupt-driven execution

Embedded Software

- Programming language
 - Programmed in **C** rather than Java
 - *smaller and faster code, so less expensive MCU*
 - Some performance-critical code may be in **assembly language**
- Operating system
 - Typically, no OS
 - *instead a **simple scheduler***
 - *or even just interrupts + main code (**foreground/background system**)*
 - If OS is used, likely to be an embedded RTOS

Hardware and Software Co-design Model

- Commonly both the hardware and the software for an embedded system are developed in parallel.
 - If performance is needed:
 - *take advantage of special hardware features.*
 - Push things to the software layer if functionality can be achieved in software:
 - *Reduces overall hardware complexity and cost.*

Functional vs. Non-functional Requirements

- Functional requirements:
 - Output as a function of input.
- Non-functional requirements:
 - Time required to compute output;
 - Size, weight, etc.;
 - *Mobile and portable*
 - Power consumption;
 - *battery capacity*
 - Reliability;
 - etc....

Functional vs. Non-functional Requirements

- Functionality is a must but embedded applications should meet many other constraints as well.
- For example, TV broadcast involves digital signal processing and cannot tolerate delays.
 - Embedded digital signal processor (DSP) has specialized arithmetic units to perform complex calculations extremely fast in real time.

Functional vs. Non-functional Requirements

- Performance goal: the program **must meet its deadline**
 - Deadline: the time at which a computation must be finished.
- If the program does not produce the required output by the deadline:
 - The program does not work properly even if the output that it eventually produces is functionally correct.

The Internet of Things (IoT)

- An Internet-of-Things system is a networked embedded computing system.
 - A large range of objects connected via a network
 - Objects can include:
 - Smart buildings and home appliances: Fridges, TVs, cookers
 - Civil engineering structures: Bridges, railways
 - Wearable devices: Smart watches, glasses
 - Medical devices: Embedded pills
- 



The Internet of Things (IoT)



Google nest
thermostat



Fitbit activity
tracker



Apple
airpods



Smart
speaker



Ring
doorbell



Xiaomi temperature
sensor



Philips hue smart
bulb

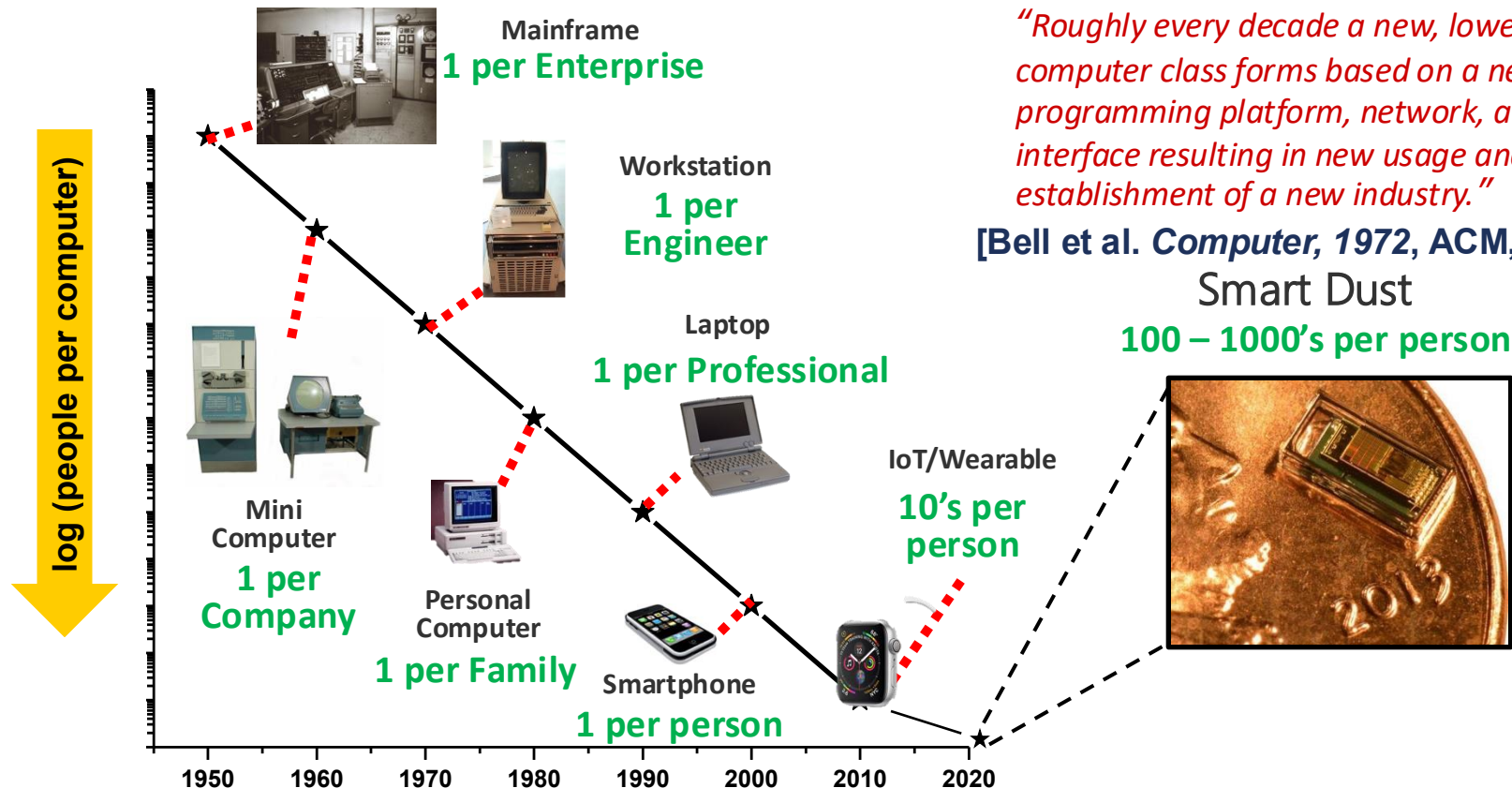


Smart
badges



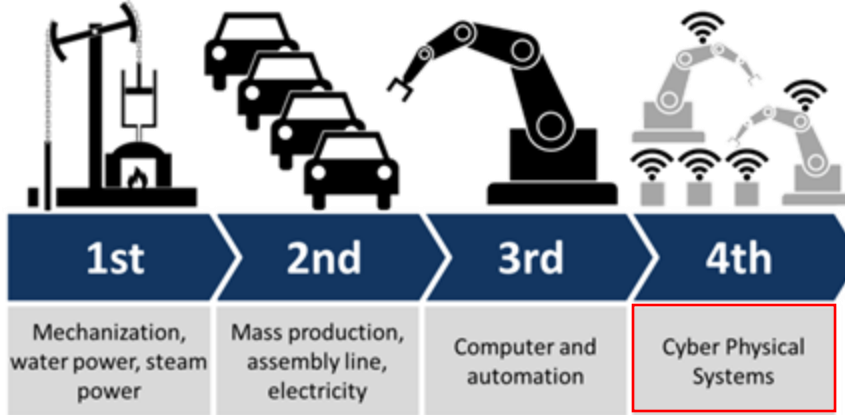
BLE proximity
beacon

Trillion embedded devices will be deployed soon



Cyber-physical Systems

- Combines physical devices with computers that control the device.
- The embedded computer is the **cyber part**
 - replaces mechanical controllers: more accurate and more sophisticated control
 - monitors and controls the physical processes with **feedback loops**



Industry 4.0

Real-time data collection, analysis, and optimization

Summary

- An embedded system is built for a specific application.
 - Complex hardware and software components.
- Embedded systems pose many design challenges:
 - Functional and non-functional requirements
 - *design time, deadlines, power, cost, etc.*
- In real-time systems, timing correctness is just as important as functional or logical correctness.